

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

**MODUL XIII
MULTI LINKED LIST**



Disusun Oleh :

NAMA : Damanik, Yohanes Geovan Ondova
NIM : 103112400022

Dosen

FAHRUDIN MUKTI WIBOWO

**PROGRAM STUDI STRUKTUR DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025**

A. Dasar Teori

Multi List: Sekumpulan *list* yang berbeda yang memiliki suatu keterhubungan satu sama lain. Tiap elemen dalam multi *linked list* dapat membentuk *list* sendiri.

Komponen: Biasanya ada yang bersifat sebagai *list* induk dan *list* anak. Dalam contoh yang diberikan, terdapat dua buah *list*, yaitu *list* pegawai sebagai induk dan *list* anak sebagai anak.

Insert:

- Insert Anak(A): Dalam penambahan elemen anak, harus diketahui dulu elemen induknya. Ilustrasi yang diberikan menggunakan konsep *insert last*..
- Insert Induk(B): Konsep *insert* elemen induk sama dengan konsep *insert* pada Singly, Doubly, dan Circular Linked List..

Delete:

- Delete Anak(A): Sama dengan *insert* anak, untuk *delete* anak harus diketahui dulu induknya.
- Delete Induk(B): Saat elemen induk dihapus, seluruh anak dengan induk tersebut juga harus dihapus.

Guided (berisi screenshot source code & output program disertai penjelasannya)

Guided 1

guided.cpp

```
#include <iostream>
#include <string>
using namespace std;

struct ChildNode
{
    string info;
    ChildNode *next;
    ChildNode *prev;
};

struct ParentNode
{
    string info;
    ChildNode *childHead;
    ParentNode *next;
    ParentNode *prev;
};

ParentNode *createParent(string info)
{
    ParentNode *newNode = new ParentNode;
    newNode->info = info;
    newNode->childHead = NULL;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}

ChildNode *createChild(string info)
{
    ChildNode *newNode = new ChildNode;
    newNode->info = info;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}

void insertParent(ParentNode *&head, string info)
{
    ParentNode *newNode = createParent(info);
    if (head == NULL)
    {
        head = newNode;
    }
}
```

```

else
{
    ParentNode *temp = head;
    while (temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}
}

void insertChild(ParentNode *head, string parentInfo, string childInfo)
{
    ParentNode *p = head;
    while (p != NULL && p->info != parentInfo)
    {
        p = p->next;
    }

    if (p != NULL)
    {
        ChildNode *newChild = createChild(childInfo);
        if (p->childHead == NULL)
        {
            p->childHead = newChild;
        } else {
            ChildNode *C = p->childHead;
            while (C->next != NULL) {
                C = C->next;
            }
            C->next = newChild;
            newChild->prev = C;
        }
    }
}

void printAll(ParentNode *head)
{
    while (head != NULL)
    {
        cout << head->info;
        ChildNode *c = head->childHead;
        while (c != NULL)
        {
            cout << " -> " << c->info;
            c = c->next;
        }
        cout << endl;
        head = head->next;
    }
}

```

```

    }
}

void updateParent(ParentNode *head, string oldInfo, string newInfo)
{
    ParentNode *p = head;
    while (p != NULL)
    {
        if (p->info == oldInfo)
        {
            p->info = newInfo;
            return;
        }
        p = p->next;
    }
}

void updateChild(ParentNode *head, string parentInfo, string oldchildInfo, string
newchildInfo)
{
    ParentNode *p = head;
    while (p != NULL && p->info != parentInfo)
    {
        p = p->next;
    }

    if (p != NULL)
    {
        ChildNode *c = p->childHead;
        while (c != NULL)
        {
            if (c->info == oldchildInfo)
            {
                c->info = newchildInfo;
                return;
            }
            c = c->next;
        }
    }
}

void deleteChild(ParentNode *head, string parentInfo, string childInfo)
{
    ParentNode *p = head;
    while (p != NULL && p->info != parentInfo)
    {
        p = p->next;
    }

    if (p != NULL)

```

```

{
    ChildNode *c = p->childHead;
    while (c != NULL)
    {
        if (c->info == childInfo)
        {
            if (c == p->childHead)
            {
                p->childHead = c->next;
                if (p->childHead != NULL)
                {
                    p->childHead->prev = NULL;
                }
            }
            else
            {
                c->prev->next = c->next;
                if (c->next != NULL)
                {
                    c->next->prev = c->prev;
                }
            }
            delete c;
            return;
        }
        c = c->next;
    }
}

```

```

void deleteParent(ParentNode *&head, string info)
{
    ParentNode *p = head;
    while (p != NULL)
    {
        if (p->info == info)
        {
            ChildNode *c = p->childHead;
            while (c != NULL)
            {
                ChildNode *tempC = c;
                c = c->next;
                delete tempC;
            }
            if (p == head)
            {
                head = p->next;
                if (head != NULL)
                {
                    head->prev = NULL;
                }
            }
        }
        p = p->next;
    }
}

```

```

    }
}
else
{
    p->prev->next = p->next;
    if (p->next != NULL)
    {
        p->next->prev = p->prev;
    }
}
delete p;
return;
}
p = p->next;
}
}

int main()
{
    ParentNode *list = NULL;
    insertParent(list, "Parent A");
    insertParent(list, "Parent B");
    insertParent(list, "Parent C");

    cout << "\nSetelah InsertParent:" << endl;
    printAll(list);

    insertClild(list, "Parent A", "Child A1");
    insertClild(list, "Parent A", "Child A2");
    insertClild(list, "Parent B", "Child B1");

    cout << "\nSetelah InsertChild:" << endl;
    printAll(list);

    updateParent(list, "Parent B", "Parent B");
    updateChild(list, "Parent A", "Child A1", "Child A1");

    cout << "\nSetelah Update:" << endl;
    printAll(list);

    deleteChild(list, "Parent A", "Child A2");
    deleteParent(list, "Parent C");

    cout << "\nSetelah Delete:" << endl;
    printAll(list);

    return 0;
}

```

Output

```
PS C:\Users\Lenovo\Documents\STRUKTUR DATA MODUL 13> cd "c:\Users\Lenovo\Documents\STRUKTUR DATA MODUL 13"
) { g++ guided.cpp -o guided } ; if ($?) { .\guided }

Setelah InsertParent:
Parent A
Parent B
Parent C

Setelah InsertChild:
Parent A -> Child A1 -> Child A2
Parent B -> Child B1
Parent C

Setelah Update:
Parent A -> Child A1 -> Child A2
Parent B -> Child B1
Parent C

Setelah Delete:
Parent A -> Child A1
Parent B -> Child B1
PS C:\Users\Lenovo\Documents\STRUKTUR DATA MODUL 13>
```

Deskripsi:

Program ini mengimplementasikan struktur data multilist (double linked list dua tingkat) yang terdiri dari parent node sebagai list utama dan child node sebagai sublist, dengan fungsi-fungsi dasar untuk menambah, mengubah, menghapus, serta menampilkan data parent dan child secara terstruktur, yang diuji pada fungsi main dengan serangkaian operasi untuk menunjukkan hubungan hierarkis antara parent dan child.

B. Unguided/Tugas (berisi screenshot source code & output program disertai penjelasannya)

Unguided 1

circularlist.cpp

```
#include "circularlist.h"
#include <iostream>
using namespace std;

void createList(List &L) {
    L.first = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = Nil;
    return P;
}

void dealokasi(address &P) {
    delete P;
}
```

```

void insertFirst(List &L, address P) {
    if (L.first == Nil) {
        P->next = P;
        L.first = P;
    } else {
        address Q = L.first;
        while (Q->next != L.first) {
            Q = Q->next;
        }
        P->next = L.first;
        Q->next = P;
        L.first = P;
    }
}

```

```

void insertAfter(List &L, address Prec, address P) {
    P->next = Prec->next;
    Prec->next = P;
}

```

```

void insertLast(List &L, address P) {
    if (L.first == Nil) {
        P->next = P;
        L.first = P;
    } else {
        address Q = L.first;
        while (Q->next != L.first) {
            Q = Q->next;
        }
        P->next = L.first;
        Q->next = P;
    }
}

```

```

void deleteFirst(List &L, address &P) {
    if (L.first != Nil) {
        P = L.first;
        if (P->next == L.first) {
            L.first = Nil;
        } else {
            address Q = L.first;
            while (Q->next != L.first) {
                Q = Q->next;
            }
            L.first = P->next;
            Q->next = L.first;
        }
        P->next = Nil;
    }
}

```

```

void deleteAfter(List &L, address Prec, address &P) {
    if (Prec != Nil) {
        P = Prec->next;
        if (P == L.first && P->next == L.first) {
            L.first = Nil;
        } else {
            Prec->next = P->next;
            if (P == L.first) {
                L.first = P->next;
            }
        }
        P->next = Nil;
    }
}

```

```

void deleteLast(List &L, address &P) {
    if (L.first != Nil) {
        if (L.first->next == L.first) {
            P = L.first;
            L.first = Nil;
        } else {
            address Q = L.first;
            while (Q->next->next != L.first) {
                Q = Q->next;
            }
            P = Q->next;
            Q->next = L.first;
        }
        P->next = Nil;
    }
}

```

```

address findElm(List L, infotype x) {
    if (L.first == Nil) {
        return Nil;
    }
}

```

```

address P = L.first;
do {
    if (P->info.nim == x.nim) {
        return P;
    }
    P = P->next;
} while (P != L.first);

return Nil;
}

```

```

void printInfo(List L) {

```

```

    if (L.first == Nil) {
        cout << "List kosong" << endl;
    } else {
        address P = L.first;
        do {
            cout << "Nama : " << P->info.nama << endl;
            cout << "NIM : " << P->info.nim << endl;
            cout << "L/P : " << P->info.jenis_kelamin << endl;
            cout << "IPK : " << P->info.ipk << endl;
            cout << endl;
            P = P->next;
        } while (P != L.first);
    }
}

```

circularlist.h

```

#ifndef CIRCULARLIST_H_INCLUDED
#define CIRCULARLIST_H_INCLUDED

#include <string>
using namespace std;

#define Nil NULL

struct infotype {
    string nama;
    string nim;
    char jenis_kelamin;
    float ipk;
};

typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L);

address alokasi(infotype x);
void dealokasi(address &P);

void insertFirst(List &L, address P);

```

```

void insertAfter(List &L, address Prec, address P);
void insertLast(List &L, address P);

void deleteFirst(List &L, address &P);
void deleteAfter(List &L, address Prec, address &P);
void deleteLast(List &L, address &P);

address findElm(List L, infotype x);

void printInfo(List L);

#endif

```

Main.cpp

```

#include "circularlist.h"
#include "circularlist.cpp"
#include <iostream>
using namespace std;

address createData(string nama, string nim, char jk, float ipk)
{
    infotype x;
    x.nama = nama;
    x.nim = nim;
    x.jenis_kelamin = jk;
    x.ipk = ipk;
    return alokasi(x);
}

void sortList(List &L) {
    if (L.first == Nil) {
        return;
    }

    address P = L.first;
    address Q;
    infotype temp;

    do {
        Q = P->next;
        while (Q != L.first) {
            if (P->info.nim > Q->info.nim) {
                temp = P->info;
                P->info = Q->info;
                Q->info = temp;
            }
            Q = Q->next;
        }
    }
}

```

```

    }
    P = P->next;
} while (P->next != L.first);
}

int main()
{
    List L;
    address P1 = Nil;
    address P2 = Nil;
    infotype x;

    createList(L);

    cout << "coba insert first, last, dan after" << endl;

    P1 = createData("Danu", "04", 'l', 4.0);
    insertFirst(L, P1);

    P1 = createData("Fahmi", "06", 'l', 3.45);
    insertLast(L, P1);

    P1 = createData("Bobi", "02", 'l', 3.71);
    insertFirst(L, P1);

    P1 = createData("Ali", "01", 'l', 3.3);
    insertFirst(L, P1);

    P1 = createData("Gita", "07", 'p', 3.75);
    insertLast(L, P1);

    x.nim = "07";
    P1 = findElm(L, x);
    P2 = createData("Cindi", "03", 'p', 3.5);
    insertAfter(L, P1, P2);

    x.nim = "02";
    P1 = findElm(L, x);
    P2 = createData("Hilmi", "08", 'l', 3.3);
    insertAfter(L, P1, P2);

    x.nim = "04";
    P1 = findElm(L, x);
    P2 = createData("Eli", "05", 'p', 3.4);
    insertAfter(L, P1, P2);

    sortList(L);

    printInfo(L);

```

```
    return 0;
}
```

Output

```
PS C:\Users\Lenovo\Documents\STRUKTUR DATA MODUL 13> cd "c:\Users\Lenovo\Documents\STRUKTUR DATA MODUL 13\UNGUIDED\"
; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
coba insert first, last, dan after
Nama : Ali
NIM : 01
L/P : 1
IPK : 3.3

Nama : Bobi
NIM : 02
L/P : 1
IPK : 3.71

Nama : Cindi
NIM : 03
L/P : p
IPK : 3.5

Nama : Danu
NIM : 04
L/P : 1
IPK : 4

Nama : Eli
NIM : 05
L/P : p
IPK : 3.4
```

```
Nama : Fahmi

Nama : Fahmi
Nama : Fahmi
NIM : 06
NIM : 06
L/P : 1
L/P : 1
IPK : 3.45
IPK : 3.45

Nama : Gita
NIM : 07
L/P : p
IPK : 3.75

Nama : Hilmi
NIM : 08
L/P : 1
IPK : 3.3
```

```
PS C:\Users\Lenovo\Documents\STRUKTUR DATA MODUL 13\UNGUIDED> █
```

Deskripsi:

Program ini mengimplementasikan Circular Linked List (CLL) untuk menyimpan data mahasiswa dengan atribut nama, NIM, jenis kelamin, dan IPK, di mana file header (circularlist.h) mendefinisikan struktur data, file implementasi (circularlist.cpp) berisi fungsi-fungsi dasar seperti penambahan, penghapusan, pencarian, dan penampilan data, sedangkan file main.cpp menguji operasi-operasi tersebut dengan menambahkan

beberapa data mahasiswa, menyisipkannya pada posisi tertentu, serta mengurutkan list berdasarkan NIM sebelum menampilkan hasilnya secara terstruktur.

C. Kesimpulan

Praktikum ini telah menerapkan dua bentuk linked list yang berbeda: Multi Linked List untuk hubungan data berjenjang dan Circular Linked List untuk pengelolaan data mahasiswa secara melingkar, dengan fungsi dasar seperti penambahan, pengubahan, dan pengurutan data. Kedua implementasi tersebut tidak hanya menunjukan cara mengorganisir data secara hierarkis dan sirkuler, tetapi juga menguatkan pemahaman terhadap operasi manipulasi data pada struktur yang lebih kompleks dalam konteks pemrograman.

D. Referensi

<https://www.geeksforgeeks.org/dsa/introduction-to-multi-linked-list/>

<https://www.geeksforgeeks.org/dsa/multilevel-linked-list/>